

The same model in scikit-learn

Everything you wrote over three weeks, in four lines. Today is not about learning a library; it is about reading one. Every argument of the constructor replaces a line you wrote by hand, and you are now one of the few people in the room who can say exactly which.

Unit 3 · Libraries

80 minutes

Lab 12, part 1

First library since session 7

fit / predict

Line against line

Why not identical

When

Lab 12 · part 1

Before Friday

PART 1 · 15 MINUTES

The fit / predict contract

One interface for hundreds of models.

1

scikit-learn is built around a promise. Every model is an object you create with its settings, `fit` on training data, and then `predict` or `score` on new data. A logistic regression, a decision tree and a neural network all look identical from the outside. That uniformity is why it is the standard tool for classical machine learning.

Lab 11, in four statements

PYTHON

```
from sklearn.neural_network import MLPClassifier

clf = MLPClassifier(hidden_layer_sizes=(16,), activation="logistic", solver="sgd",
                    learning_rate_init=1.0, batch_size=32, max_iter=40,
                    momentum=0.0, alpha=0.0, random_state=7)
clf.fit(X_train, y_train.ravel())
print(clf.score(X_test, y_test.ravel()))
```

output

OUTPUT

0.936

AN API DESIGNED FOR PEOPLE WHO ALREADY UNDERSTAND THE THING

That call is only readable because you wrote its insides. `activation="logistic"` is your sigmoid. `solver="sgd"` with `momentum=0.0` is your update line. `batch_size=32` and `max_iter=40` are your two loops. Someone meeting this call first learns a list of settings; you are reading a program.

PART 2 · 25 MINUTES

Every removed line, named

Put the Lab 11 notebook on the left and the four lines on the right.

2

Your code	Where it went in scikit-learn
<code>init_params</code>	<code>fit</code> , using Glorot initialisation and <code>random_state</code> — not your $N(0, 0.1)$
<code>sigmoid</code> , <code>forward</code>	<code>hidden_layer_sizes</code> and <code>activation</code> ; the output sigmoid is chosen for you because the labels are binary
<code>bce</code>	built in: <code>MLPClassifier</code> always uses cross-entropy
<code>backward</code>	inside <code>fit</code> — the same four matrix lines, in the library's source
<code>p[k] -= lr * g[k]</code>	<code>solver="sgd"</code> , <code>learning_rate_init</code> , <code>momentum=0.0</code>
shuffle and batch slice	<code>shuffle=True</code> (the default) and <code>batch_size</code>
epoch loop	<code>max_iter</code>
<code>accuracy</code>	<code>score</code>



Defaults are decisions someone else made

Leave out `solver` and you get Adam, not SGD. Leave out `alpha` and you get a weight penalty of 0.0001. Leave out `momentum` and it is 0.9. None of these is wrong — but a result you cannot reproduce by hand is a result you cannot debug. Today we switch every one of them off on purpose, then switch them back on knowingly.

PART 3 · 15 MINUTES

Close, but not identical

97.6% by hand, 93.6% by library. Why?

3

Same architecture, same learning rate, same batch size, same number of epochs — and four points of accuracy apart. The difference is not a bug on either side. It is everything the settings did *not* pin down:

- **Starting weights.** scikit-learn draws from a Glorot uniform range scaled to each layer's width; you drew from $N(0, 0.1)$. Different starting points, different walks.
- **Batch order.** Both shuffle every epoch, but from different random generators, so the 1,720 updates happen in a different sequence.
- **Small internals.** Details such as how the loss is averaged and when the last partial batch is used.

THE LESSON IN THE GAP

Two correct implementations of the same model, trained the same way, can differ by several points on 450 test images. **A single run's accuracy is a sample, not a fact.** Before claiming one model is better than another, ask whether the gap is larger than the difference a new random seed would make.

PART 4 · 10 MINUTES

When to use which

4



Write it yourself

To understand a method, to check a library, or when you need something the library does not offer.

scikit-learn

Tabular data, classical models, pipelines, evaluation. Sessions 20–22 live here.



PyTorch / Keras

Deep networks, GPUs, custom architectures, images and text. Friday.

LAB 12, PART 1 · 20 MINUTES IN CLASS

Reproduce Lab 11 with MLPClassifier

5

Lab 12 — `CSC4315_Lab12_One_Model_Three_Frameworks`, Part 1. Fill the constructor from Lab 11's settings, fill the line–against–line table, and name two reasons the accuracies differ. Part 2 is Friday, in the same notebook.

Before Friday

1. **Finish Lab 12, part 1.**
2. **Check that PyTorch and TensorFlow import** in `atd_env` — or open the notebook on Colab. Friday needs both.



Session 18 — Friday 06 November, 11:30

PyTorch and Keras, side by side — started from identical weights, so that three frameworks must produce one loss curve.